

Lab Manual: Working with Data in Streamlit

Objective

By the end of this lab, you will:

- Upload and read datasets into a Streamlit app.
- Perform **data cleaning** interactively.
- Run quick **statistical analysis** on the data.
- Create **visualizations** inside Streamlit.

Prerequisites

- Python 3.7+ installed.
- Streamlit and Matplotlib installed: `pip install streamlit pandas matplotlib`

Step 1: Create a New Streamlit App

Create a file called `data_app.py` and start with:

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.title("Working with Data in Streamlit")
```

Explanation:

- `streamlit` → builds the interactive UI.
- `pandas` → handles data loading and analysis.
- `matplotlib` → generates visualizations.
- `st.title()` → sets the app's title at the top.

Step 2: Upload a CSV File

```
uploaded_file = st.file_uploader("Upload your CSV file", type=["csv"])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.write("Preview of Data:")
    st.dataframe(df.head())
```

Explanation:

- `st.file_uploader()` → creates a file upload widget.
- If a file is uploaded, it's read into a Pandas DataFrame.
- `st.dataframe(df.head())` → shows the first 5 rows interactively.

Step 3: Clean the Data

```
if uploaded_file is not None:
    if st.checkbox("Show missing value counts"):
        st.write(df.isnull().sum())

    if st.checkbox("Drop rows with missing values"):
        df = df.dropna()
        st.write("Data after dropping missing values:")
        st.dataframe(df.head())
```

Explanation:

- `df.isnull().sum()` → counts missing values per column.
- `st.checkbox()` → toggles options for showing missing values or dropping them.
- `df.dropna()` → removes rows with missing data.

Step 4: Quick Analysis

```
if uploaded_file is not None:
    st.subheader("Quick Statistics")
```

```
st.write(df.describe())
```

Explanation:

- `df.describe()` → generates summary statistics (mean, min, max, etc.).
- `st.write()` → displays them neatly in the app.

Step 5: Choose a Column for Visualization

```
if uploaded_file is not None:
    numeric_columns =
df.select_dtypes(include="number").columns.tolist()

    if numeric_columns:
        selected_col = st.selectbox("Choose a numeric column for
visualization", numeric_columns)
```

Explanation:

- `df.select_dtypes(include="number")` → selects numeric columns only.
- `st.selectbox()` → lets the user pick one numeric column from a dropdown.

Step 6: Visualise the Data

```
# Create histogram
fig, ax = plt.subplots()
ax.hist(df[selected_col], bins=20, color="skyblue",
edgecolor="black")
ax.set_title(f"Histogram of {selected_col}")

# Render in Streamlit
st.pyplot(fig)
```

Explanation:

- `plt.subplots()` → creates a Matplotlib figure and axes.

- `ax.hist()` → plots a histogram of the chosen column.
- `st.pyplot(fig)` → displays the Matplotlib figure in Streamlit.

Step 7: Run the App

In your terminal: `streamlit run data_app.py`

Then open <http://localhost:8501> in your browser.

Verification Checklist

- App title appears.
- File uploader accepts CSV.
- Data preview shows first rows.
- Missing value counts display correctly.
- Dropping missing values updates the DataFrame.
- `df.describe()` statistics visible.
- Dropdown lists numeric columns.
- Histogram renders for the chosen column.

Summary

In this lab, you built an **interactive data workflow app**:

1. **Upload** → Load CSV files into Pandas.
2. **Clean** → Check & drop missing values.
3. **Analyze** → Generate quick summary statistics.
4. **Visualize** → Plot histograms interactively.

With just a few lines of code, you now have a **browser-based data exploration tool** that updates instantly with user input.